

Step 5: Deployment and Simulation

BCS3210 Blockchains group project: Decentralized Digital Identity and Data Sharing Platform

What we did in this step

We deployed the contracts to a local Hardhat node (`npx hardhat node`, chain id 31337) with Hardhat Ignition, and wrote a simulation script (`scripts/simulate.ts`) that lets many users with different roles use the platform at the same time. We ran it for 5, 10, 20, 50 and 100 patients and recorded gas and confirmation time for every operation. We did not deploy to a public testnet (optional in the brief); instead we compared the local node with Hardhat's in-process network.

1. Deployment

```
npx hardhat node          # terminal 1
npm run deploy:local      # terminal 2, Ignition module PlatformModule
```

Ignition deploys `AccessToken`, `DigitalIdentity`, `ConsentManager(identity, token)` and `DataSharing(identity, consentManager)`, then calls `setMinter`. The addresses are saved in `ignition/deployments/chain-31337/deployed_addresses.json`, which the Python vault and the front-end read. After the deployment we ran the off-chain demo (`offchain/demo.py`) against the node again to check everything still works with the final contracts.

2. Simulation setup

For every run the script deploys new contracts and creates new random accounts (funded with test ETH):

- 1 admin (Hardhat account #0), **N patients** and **N/5 requesters** (at least 1).
- Admin approves all requesters, every patient registers and gives consent (scope Both, 30 days) to one requester.
- Every requester requests access to its patients (granted).
- Half of the patients revoke, then every requester requests again (half denied, half granted).
- At the end we read the full audit trail back from the `AccessGranted` and `AccessDenied` events.

Time is measured from sending the transaction until the receipt is available (confirmation), with `performance.now()`. Transactions are sent one after the other.

```
npx hardhat run scripts/simulate.ts --network localhost
```

3. Results on the local node

Totals per run

Patients	Requesters	Transactions	Total gas	Total time (s)	Throughput (tx/s)	Events read	Event read time (ms)
5	1	24	2,143,847	0.27	88.0	10	7.8
10	2	47	4,224,675	0.47	99.4	20	3.8
20	4	94	8,415,198	0.85	110.2	40	5.1

Patients	Requesters	Transactions	Total gas	Total time (s)	Throughput (tx/s)	Events read	Event read time (ms)
50	10	235	20,987,175	1.97	119.1	100	16.0
100	20	470	41,939,790	3.80	123.6	200	27.3

Average gas per operation

Operation	5	10	20	50	100
approveRequester	49,680	49,692	49,689	49,692	49,690
registerUser	147,552	145,845	144,996	144,486	144,316
grantConsent	96,533	94,835	93,977	93,467	93,294
requestAccess (1st, granted)	87,207	87,207	87,208	87,209	87,208
revokeConsent	28,957	28,969	28,963	28,969	28,967
requestAccess (2nd, denied)	70,207	70,206	70,211	70,210	70,210
requestAccess (2nd, granted)	70,109	70,109	70,108	70,109	70,108

Average confirmation time per operation (ms)

Operation	5	10	20	50	100
approveRequester	6.0	8.2	4.9	5.0	4.3
registerUser	12.9	9.7	8.6	9.1	7.9
grantConsent	11.5	13.2	9.6	8.9	9.3
requestAccess (1st, granted)	10.6	8.7	8.4	8.1	8.1
revokeConsent	11.3	8.6	8.5	7.0	7.1
requestAccess (2nd, denied)	10.6	9.8	10.8	8.2	8.4
requestAccess (2nd, granted)	12.6	9.7	10.7	9.5	8.2

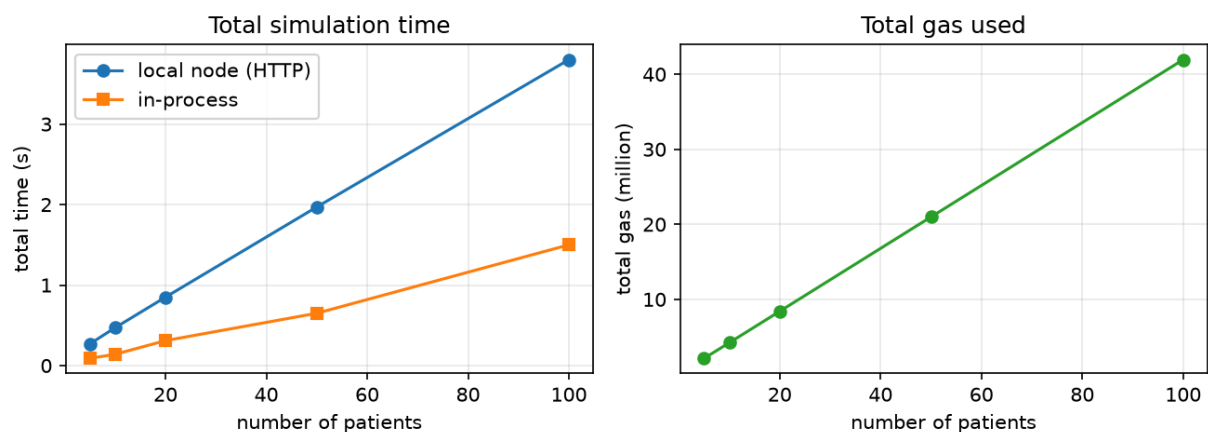


Figure: total time and total gas grow linearly with the number of patients.

4. Local node vs in-process network

Patients	Local node total (s)	In-process total (s)	Local node tx/s	In-process tx/s
5	0.27	0.09	88.0	257.8

Patients	Local node total (s)	In-process total (s)	Local node tx/s	In-process tx/s
10	0.47	0.14	99.4	344.9
20	0.85	0.31	110.2	303.5
50	1.97	0.65	119.1	361.6
100	3.80	1.50	123.6	313.2

Gas is the same on both (it only depends on the contract code). The in-process network is about 3 times faster because there is no HTTP/JSON-RPC between the script and the chain.

5. Analysis

- **Cost scales linearly.** Gas per operation does not depend on the number of users: every function only touches the storage of one patient/requester pair and has no loops over all users. The whole lifecycle of one patient (register, consent, two requests, revoke for half) costs about 420,000 gas, for 5 patients and for 100 patients alike. Total gas grows exactly with N.
- **Time scales linearly** as well: 3.8 s for 470 transactions. Confirmation time per transaction stays around 8 to 10 ms (the first runs are a bit slower while the node warms up). Throughput is about 90 to 125 transactions per second, limited by sending them one after the other over HTTP.
- **Reading the audit log** is cheap and fast: 200 events in 27 ms, and it costs no gas because it is a read.
- **Real network.** The local node mines every transaction immediately. On Ethereum mainnet a block comes every 12 seconds, so a request would take at least 12 seconds to confirm, and the cost depends on the gas price. At 20 gwei, one granted request (70,108 gas) would cost about 0.0014 ETH. The number of requests per block is limited by the block gas limit, which is why the optimisation in Step 4 (-35% on requestAccess) matters.